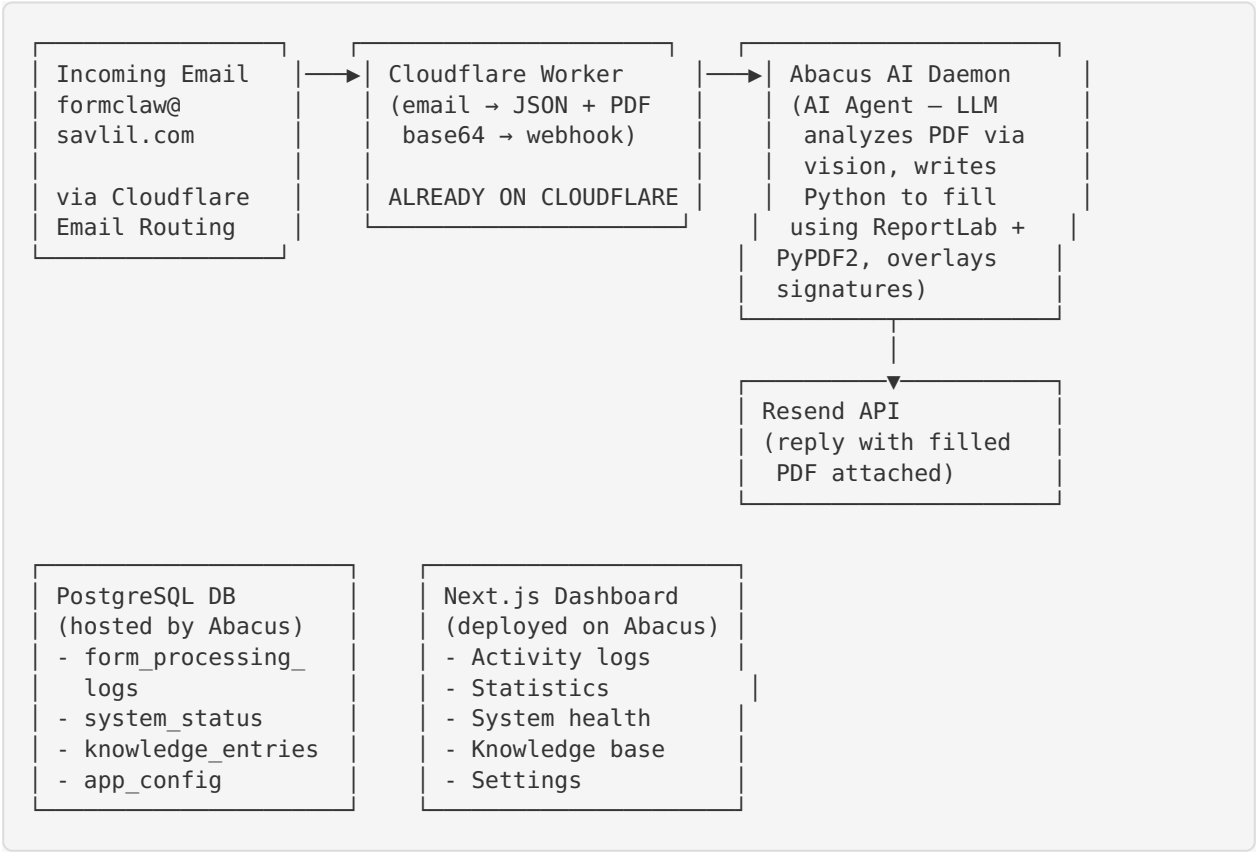
Form Claw — Cloudflare Workers Migration Plan

Date: 2026-07-09
Author: Form Claw Project
Status: DRAFT — awaiting review

---

1. Current Architecture (Abacus AI + Cloudflare hybrid)



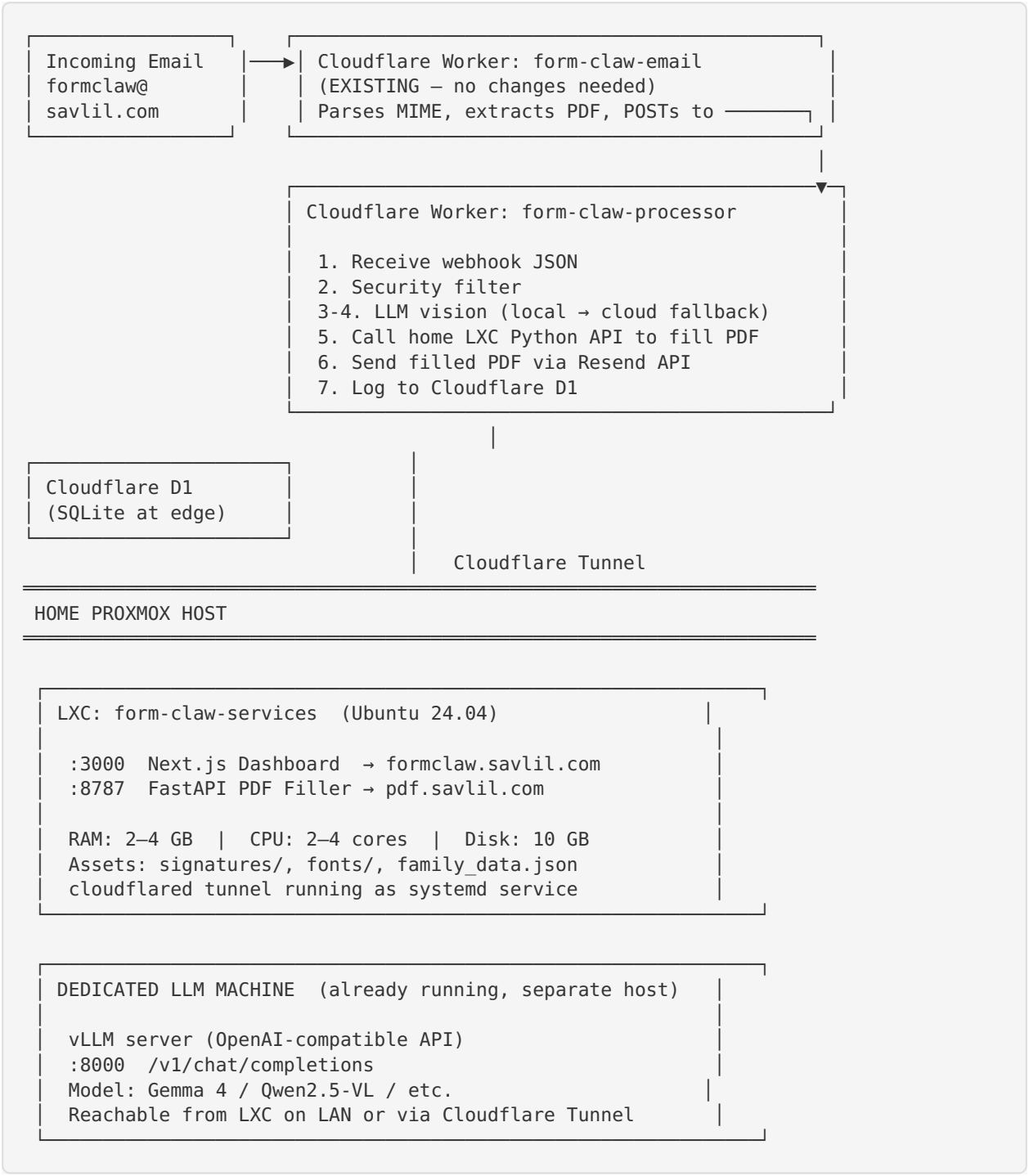
Key insight: The “brain” is an LLM agent

The form processor is NOT a deterministic script. It is an **AI agent** that:
1. Receives a PDF (base64) in the webhook payload
2. Uses **LLM vision** to analyze the PDF form (identify field labels, coordinates, checkboxes)
3. Maps fields to family data from the knowledge base
4. **Writes and executes Python code** at runtime using ReportLab + PyPDF2 to:
- Place text at precise pixel coordinates
- Handle Hebrew RTL text
- Split ID numbers into individual digit boxes
- Draw ellipses around selected options
- Overlay transparent signature PNGs
5. Returns the filled PDF

This means the migration is NOT about porting a fixed codebase — it's about:
- Providing an LLM API that supports **vision** (PDF page → image analysis)
- Providing a **code execution sandbox** (Python with ReportLab, PyPDF2, Pillow)
- Providing **storage** for signatures, fonts, family data, and output PDFs

---

2. Target Architecture (Cloudflare + self-hosted Proxmox)



LXC vs VM: What Goes Where

Service	Where	Why
FastAPI PDF Filler	✅ LXC	Pure Python, no hardware access needed. Lowest overhead.
Next.js Dashboard	✅ LXC	Node.js process, no special kernel features.
Cloudflare Tunnel	✅ LXC	Userspace binary, works everywhere.
vLLM (self-hosted LLM)	✅ **Existing dedicated machine**	Already running — no changes needed, reachable on LAN.

Recommendation: Run PDF filler + dashboard + cloudflared in a **single LXC container** (2–4 vCPU, 2–4 GB RAM, 10 GB disk). The LLM stays on your existing dedicated machine — the Cloudflare Worker (or the LXC services) calls it over your LAN or via Tunnel.

---

3. Component-by-Component Migration

3.1 Email Intake Worker (form-claw-email)

Status: ✅ Already on Cloudflare
Changes needed: Point `WEBHOOK_URL` to the new processor worker instead of Abacus daemon
Effort: Config change only (update Cloudflare secret)

3.2 Form Processor Worker (form-claw-processor) — THE HARD PART

This is the core challenge. The current processor is an AI agent with:
- Full Python runtime (ReportLab, PyPDF2, Pillow, tesseract)
- Multi-step LLM reasoning with vision
- File system access (read/write PDFs, PNGs)

Cloudflare Workers limitations:
- No Python runtime (Workers run V8/JavaScript/WASM)
- 30-second CPU time limit (free) / 15-minute wall-clock (paid, with Cron Triggers)
- No file system — must use R2 for storage
- Memory limit: 128MB

Proposed approach: The Worker becomes an **orchestrator** that calls your home VM + LLM API:

```
form-claw-processor (JS/TS Worker)
  │
  ├─► LLM API (vision + reasoning)
  │     • Analyze PDF pages as images
  │     • Determine field coordinates & values
  │     • Generate fill instructions JSON
  │
  ├─► Home VM Python API (self-hosted on Proxmox)
  │     • Receives fill instructions + PDF + signature refs
  │     • Executes ReportLab/PyPDF2 fill
  │     • Returns filled PDF base64
  │     • Assets (signatures, fonts) stored locally on the VM
  │
  ├─► R2 (Cloudflare object storage)
  │     • Store filled PDFs temporarily (optional backup)
  │
  ├─► Resend API (send reply — already working)
  │
  └─► Cloudflare D1 (SQLite)
        • Log processing events
        • System status
        • Knowledge base
```

3.3 Database — Cloudflare D1

Current: Abacus-hosted PostgreSQL
Target: Cloudflare D1 (https://developers.cloudflare.com/d1/) (SQLite at the edge)
Why D1: Native Cloudflare integration — zero latency from Workers, no external connection strings, free tier generous (5GB storage, 5M reads/day, 100K writes/day), zero cold-start

Migration work:
- Rewrite Prisma schema → D1 SQL schema (SQLite dialect: no `@db.Text`, no enums, `REAL` instead of `Decimal`, `TEXT` for DateTime stored as ISO strings)
- Replace Prisma ORM calls in the Worker with D1 prepared statements (direct `env.DB.prepare()`)
- Dashboard can either:
- (a) Keep Prisma and read from D1 via the Cloudflare REST API (`/client/v4/accounts/.../d1/database/.../query`), or
- (b) Stay on Abacus Postgres and sync data via a lightweight D1→Postgres replication (a Cron Trigger that pushes new rows)

D1 Schema (SQLite equivalent of current Prisma models):

```
CREATE TABLE form_processing_log (
  id TEXT PRIMARY KEY DEFAULT (lower(hex(randomblob(16)))),
  sender_email TEXT,
  sender_name TEXT,
  subject TEXT,
  received_at TEXT DEFAULT (datetime('now')),
  processing_status TEXT DEFAULT 'pending',
  processing_time_seconds REAL,
  target_person TEXT,
  error_type TEXT,
  error_message TEXT,
  filled_pdf_url TEXT,
  original_pdf_url TEXT,
  ai_notes TEXT
);

CREATE TABLE system_status (
  id INTEGER PRIMARY KEY DEFAULT 1,
  email_source TEXT DEFAULT 'cloudflare',
  webhook_enabled INTEGER DEFAULT 1,
  form_process_status TEXT DEFAULT 'ok',
  last_form_processed TEXT,
  last_error_at TEXT,
  last_cloudflare_email TEXT,
  updated_at TEXT DEFAULT (datetime('now'))
);

CREATE TABLE knowledge_entry (
  id TEXT PRIMARY KEY DEFAULT (lower(hex(randomblob(16)))),
  key TEXT NOT NULL,
  value TEXT NOT NULL,
  category TEXT DEFAULT 'general',
  language TEXT DEFAULT 'he',
  applies_to_person TEXT,
  source TEXT DEFAULT 'manual',
  is_active INTEGER DEFAULT 1,
  created_at TEXT DEFAULT (datetime('now')),
  updated_at TEXT DEFAULT (datetime('now'))
);

CREATE TABLE app_config (
  id TEXT PRIMARY KEY DEFAULT (lower(hex(randomblob(16)))),
  key TEXT UNIQUE NOT NULL,
  value TEXT NOT NULL,
  label TEXT,
  category TEXT DEFAULT 'general',
  updated_at TEXT DEFAULT (datetime('now'))
);

CREATE INDEX idx_log_received ON form_processing_log(received_at);
CREATE INDEX idx_log_status ON form_processing_log(processing_status);
CREATE INDEX idx_knowledge_active ON knowledge_entry(is_active, category);
```

3.4 Dashboard — Self-Hosted on Proxmox VM (recommended)

Run the Next.js dashboard on the same Proxmox host as the PDF filler, exposed via Cloudflare Tunnel.

Why self-host

- Full control, no platform dependency
- Same VM or sibling LXC — negligible extra resource cost

- Direct access to D1 via HTTP API (or local SQLite replica)
- Cloudflare Tunnel = free TLS, no port forwarding

Dashboard runs in the same LXC

No separate container needed — the dashboard runs alongside the PDF filler in the same LXC (port 3000 for dashboard, port 8787 for PDF API). Node.js 20 is already installed for the dashboard.

Database Strategy

The dashboard needs to read the same data the Processor Worker writes to D1. Options:

Approach	Pros	Cons
D1 HTTP API (recommended)	Real-time reads, single source of truth, no sync lag	Requires Cloudflare API token, REST calls from server-side
Local SQLite replica	Zero network latency, works offline	Need sync mechanism (Litestream or cron-based D1 export)
D1 + local write-through	Dashboard can also write (knowledge entries, config)	More complex, but fully self-contained

Recommended: Use the **D1 HTTP REST API** from the dashboard's API routes. Replace Prisma calls with a thin D1 client:

```
// lib/d1-client.ts
const D1_API = `https://api.cloudflare.com/client/v4/accounts/${pro-
cess.env.CF_ACCOUNT_ID}/d1/database/${process.env.D1_DATABASE_ID}/query`;

export async function queryD1(sql: string, params: any[] = []) {
  const res = await fetch(D1_API, {
    method: 'POST',
    headers: {
      'Authorization': `Bearer ${process.env.CF_API_TOKEN}`,
      'Content-Type': 'application/json'
    },
    body: JSON.stringify({ sql, params })
  });
  const data = await res.json();
  return data.result[0].results;
}
```

Setup Steps

```
# Inside the LXC (or same container as PDF filler):

# 1. Install Node.js 20
curl -fsSL https://deb.nodesource.com/setup_20.x | bash -
apt install -y nodejs

# 2. Clone repo and build
cd /opt
git clone https://github.com/zeevrussak/form-claw.git
cd form-claw/nextjs_space

# 3. Create .env for self-hosted mode
cat > .env << 'EOF'
# D1 access (replaces Prisma/Postgres)
CF_ACCOUNT_ID=<your-cloudflare-account-id>
D1_DATABASE_ID=<your-d1-database-id>
CF_API_TOKEN=<cloudflare-api-token-with-d1-read-write>

# Auth
NEXTAUTH_URL=https://formclaw.savlil.com
NEXTAUTH_SECRET=<generate-with-openssl-rand-hex-32>
GOOGLE_CLIENT_ID=<your-google-oauth-client-id>
GOOGLE_CLIENT_SECRET=<your-google-oauth-client-secret>

# Optional: Resend key for health checks
RESEND_API_KEY=<your-resend-key>
EOF

# 4. Install deps and build
npm install  # or yarn
npm run build

# 5. Create systemd service
cat > /etc/systemd/system/form-claw-dashboard.service << 'EOF'
[Unit]
Description=Form Claw Dashboard
After=network.target

[Service]
Type=simple
WorkingDirectory=/opt/form-claw/nextjs_space
ExecStart=/usr/bin/node .next/standalone/server.js
Restart=always
RestartSec=5
Environment=PORT=3000
Environment=NODE_ENV=production

[Install]
WantedBy=multi-user.target
EOF

systemctl enable --now form-claw-dashboard
```

Cloudflare Tunnel Ingress (add to existing tunnel config)

```
# /etc/cloudflared/config.yml — add this ingress rule:
ingress:
  - hostname: formclaw.savlil.com
    service: http://localhost:3000
  - hostname: pdf.savlil.com
    service: http://localhost:8787
  - service: http_status:404
```

Result: `https://formclaw.savlil.com` serves the dashboard, `https://pdf.savlil.com` serves the PDF API — both from the same VM, same tunnel.

Migration from Prisma to D1

The current dashboard uses Prisma with PostgreSQL. For self-hosting with D1, you'll need to:
1. Replace `prisma` imports with the D1 HTTP client in all API routes
2. Rewrite queries from Prisma syntax to raw SQL (the D1 schema in Section 3.3 maps 1:1)
3. Keep NextAuth — it works fine with a local SQLite file for session/user storage, or use Cloudflare Access instead
4. The dashboard UI code (React components, charts) stays identical — only the data-fetching layer changes

Estimated effort: 1-2 days for the Prisma → D1 migration.

3.5 Health Monitoring

- The `/api/health/check` route stays with the dashboard (wherever it lives)
- The Warning Investigator can run as a Cloudflare Cron Trigger
- Alerts continue via Resend API

---

4. LLM API Options

The form processor needs:

1. **Vision capability**: Send PDF pages as images, get field analysis
2. **Reasoning**: Multi-step thinking to map family data to detected fields
3. **Structured output**: Return JSON with field coordinates and values
4. **Hebrew language support**: Understand Hebrew form labels

Option A: Abacus AI ChatLLM API (current)

Endpoint: `https://apps.abacus.ai/api/v0/chat/completions`
Key: `ABACUSAI_API_KEY` (already configured)
Models: Routes to best available (GPT-4o, Claude, Gemini, etc.)
Vision: ✅ Supported via `modalities: ["image"]` or base64 image in messages
Hebrew: ✅ Excellent
Cost: Included in Abacus subscription (you're already paying)

Pros:
- Already configured and working
- No additional API key needed
- Model routing optimizes cost/quality

Cons:
- Dependency on Abacus AI platform
- Can’t use if you leave Abacus entirely

Integration from Cloudflare Worker:

```
const response = await fetch('https://apps.abacus.ai/api/v0/chat/completions', {
  method: 'POST',
  headers: {
    'Authorization': `Bearer ${env.ABACUSAI_API_KEY}`,
    'Content-Type': 'application/json'
  },
  body: JSON.stringify({
    messages: [
      {
        role: 'user',
        content: [
          { type: 'text', text: 'Analyze this PDF form...' },
          { type: 'image_url', image_url: { url: `${pageImage-
Base64}` } }
        ]
      }
    ],
    model: 'gpt-4o'  // or let RouteLLM choose
  })
});
```

Option B: xAI Grok API

Endpoint: `https://api.x.ai/v1/chat/completions`
Key: Get from console.x.ai (https://console.x.ai)
Models: `grok-2-vision-1212` (vision), `grok-2-1212` (text)
Vision: ✅ Supported (OpenAI-compatible format)
Hebrew: ✅ Good (not as extensively tested as GPT-4o for Hebrew)

Pricing (as of 2026):
Model	Input	Output
grok-2-vision	$2.00 / 1M tokens	$10.00 / 1M tokens
grok-2	$2.00 / 1M tokens	$10.00 / 1M tokens

Integration from Cloudflare Worker:

```
const response = await fetch('https://api.x.ai/v1/chat/completions', {
  method: 'POST',
  headers: {
    'Authorization': `Bearer ${env.XAI_API_KEY}`,
    'Content-Type': 'application/json'
  },
  body: JSON.stringify({
    model: 'grok-2-vision-1212',
    messages: [
      {
        role: 'user',
        content: [
          { type: 'text', text: 'Analyze this PDF form...' },
          { type: 'image_url', image_url: { url: `${pageImage-
Base64}` } }
        ]
      }
    ]
  })
});
```

Pros:
- OpenAI-compatible API (easy swap)
- Competitive pricing
- No platform lock-in

Cons:
- Requires separate API key & billing
- Hebrew OCR quality may vary vs GPT-4o
- Grok vision is newer, less battle-tested for precise coordinate extraction

Option C: Self-Hosted LLM via vLLM (your dedicated LLM machine)

You already have a dedicated machine running **vLLM** with an OpenAI-compatible API. This is the primary LLM provider — zero API costs, zero external dependencies, already operational.

Recommended Models

Model	Size	VRAM (Q4)	Vision	Hebrew	License	Notes
Gemma 4 12B	12B	~8 GB	✅ Native (encoder-free)	⚠️ Moderate	Apache 2.0	Best balance of speed + quality for a single GPU
Gemma 4 27B	31B dense	~18 GB	✅ Native + bounding boxes	⚠️ Moderate	Apache 2.0	Best open model for document analysis; needs 24GB GPU
Gemma 4 26B MoE	26B (3.8B active)	~15 GB	✅ Native	⚠️ Moderate	Apache 2.0	Fast inference via sparse activation
Qwen2.5-VL-7B	7B	~5 GB	✅ Excellent OCR	✅ Good	Apache 2.0	Smallest viable option; strong document understanding
Qwen2.5-VL-72B	72B	~42 GB	✅ Best-in-class OCR	✅ Good	Apache 2.0	Enterprise quality but needs serious hardware
Phi-4 Reasoning Vision	14B	~8 GB	✅ Good	⚠️ Limited	MIT	Strong reasoning, smaller footprint

Model Selection Guide

Pick the right model for your dedicated vLLM machine based on its GPU:

Model	VRAM needed (Q4)	Vision Quality	Hebrew Quality	Notes
Gemma 4 12B	~8 GB	✅ Good	⚠️ Moderate	Best speed/quality ratio
Gemma 4 27B	~18 GB	✅ Excellent + bounding boxes	⚠️ Moderate	Best open model for doc analysis
Qwen2.5-VL-7B	~5 GB	✅ Excellent OCR	✅ Good	Best Hebrew OCR among smaller models
Qwen2.5-VL-72B	~42 GB	✅ Best-in-class	✅ Good	If your machine has the VRAM

> **Recommendation for Hebrew forms**: Start with **Qwen2.5-VL-7B** (best Hebrew OCR in its class). If your machine has 24+ GB VRAM, try **Gemma 4 27B** for superior document analysis.

vLLM Setup (already running)

Your dedicated LLM machine already runs vLLM. Ensure it's configured with a vision-capable model and the OpenAI-compatible API enabled:

```
# vLLM serves an OpenAI-compatible API (default port 8000)
# Typical startup command:
vllm serve google/gemma-4-12b-it \  # or Qwen/Qwen2.5-VL-7B-Instruct
  --port 8000 \
  --api-key "your-token"  # optional, for auth

# Verify vision works:
curl http://<LLM_MACHINE_IP>:8000/v1/chat/completions \
  -H "Content-Type: application/json" \
  -H "Authorization: Bearer your-token" \
  -d '{
    "model": "google/gemma-4-12b-it",
    "messages": [{
      "role": "user",
      "content": [
        {"type": "text", "text": "Analyze this PDF form..."},
        {"type": "image_url", "image_url": {"url": "..."}}
      ]
    }]
  }'
```

Exposing vLLM to Cloudflare Worker

The Cloudflare Worker needs to reach your vLLM server. Two approaches:

Approach A: Cloudflare Tunnel (recommended — if LLM machine has cloudflared)

```
# On the LLM machine, add to cloudflared config:
ingress:
  - hostname: llm.savlil.com
    service: http://localhost:8000
```

Then in the Worker: `LLM_API_URL = https://llm.savlil.com/v1/chat/completions`

Approach B: Route through the LXC (no extra tunnel needed)
The Worker calls `pdf.savlil.com` (the LXC), and the PDF filler LXC calls vLLM over LAN (`http://<LLM_IP>:8000`). This keeps the LLM API fully private — never exposed to the internet.

Approach C: Direct tunnel from existing LXC
Add the LLM machine's IP as an ingress in the LXC's existing cloudflared tunnel:

```
ingress:
  - hostname: llm.savlil.com
    service: http://<LLM_MACHINE_LAN_IP>:8000
```

This works if the LXC can reach the LLM machine on your LAN.

Integration from Cloudflare Worker

```
// Same OpenAI-compatible format as ChatLLM and Grok — vLLM is fully compatible:
const response = await fetch(env.LLM_API_URL, {  // https://llm.savlil.com/v1/chat/
completions
  method: 'POST',
  headers: {
    'Content-Type': 'application/json',
    'Authorization': `Bearer ${env.VLLM_API_KEY}`  // if vLLM requires auth
  },
  body: JSON.stringify({
    model: env.LLM_MODEL,  // e.g. 'google/gemma-4-12b-it'
    messages: [{
      role: 'user',
      content: [
        { type: 'text', text: 'Analyze this Hebrew PDF form...' },
        { type: 'image_url', image_url: { url: `${pageImage-
Base64}` } }
      ]
    }]
  })
});
```

Risks of Self-Hosted LLM

Risk	Impact	Mitigation
Hebrew OCR quality lower than GPT-4o	Field misidentification	Test with 10+ real Hebrew forms before committing; fall back to ChatLLM/Grok for Hebrew-heavy forms
Slower inference (~5-30s per vision call)	Longer form processing	Acceptable for 2/day; use quantized model; GPU passthrough
Model updates require manual action	Miss improvements	Update model weights periodically; vLLM supports hot-reload with `--served-model-name`
LLM machine down (maintenance, power)	LLM unavailable	Worker auto-falls back to ChatLLM/Grok; LLM machine has its own UPS
Power consumption	Higher electricity	GPU idle power is ~15-30W; machine is already running for other tasks

LLM Cascade: Self-Hosted First, Cloud Fallback

The Worker tries the self-hosted LLM first. If it’s unreachable or returns an error, it automatically retries with the configured cloud fallback (ChatLLM or Grok):

```
// src/llm.ts
type LLMProvider = 'chatllm' | 'grok' | 'selfhosted';

interface LLMConfig {
  url: string;
  headers: Record<string, string>;
  model: string;
}

function getProviderConfig(provider: LLMProvider, env: Env): LLMConfig {
  switch (provider) {
    case 'selfhosted':
      return {
        url: env.LLM_API_URL,   // https://llm.savlil.com/v1/chat/completions
        headers: env.VLLM_API_KEY ? { 'Authorization': `Bearer ${env.VLLM_API_KEY}
` } : {},
        model: env.LLM_MODEL || 'google/gemma-4-12b-it'
      };
    case 'chatllm':
      return {
        url: 'https://apps.abacus.ai/api/v0/chat/completions',
        headers: { 'Authorization': `Bearer ${env.LLM_API_KEY}` },
        model: 'gpt-4o'
      };
    case 'grok':
      return {
        url: 'https://api.x.ai/v1/chat/completions',
        headers: { 'Authorization': `Bearer ${env.LLM_API_KEY}` },
        model: 'grok-2-vision-1212'
      };
  }
}

/** Try self-hosted first; on failure, fall back to cloud provider. */
async function callLLM(messages: any[], env: Env): Promise<{ result: any; provider: st
ring }> {
  const primary = getProviderConfig('selfhosted', env);
  const fallback = getProviderConfig(
    (env.LLM_FALLBACK_PROVIDER as LLMProvider) || 'chatllm', env
  );

  // 1. Try self-hosted
  try {
    const res = await fetch(primary.url, {
      method: 'POST',
      headers: { 'Content-Type': 'application/json', ...primary.headers },
      body: JSON.stringify({ model: primary.model, messages }),
      signal: AbortSignal.timeout(120_000)  // 2 min timeout for local inference
    });
    if (res.ok) {
      const data = await res.json();
      return { result: data, provider: `selfhosted/${primary.model}` };
    }
    console.log(`Self-hosted LLM returned ${res.status}, falling back...`);
  } catch (err) {
    console.log(`Self-hosted LLM unreachable: ${err}, falling back...`);
  }

  // 2. Fall back to cloud
  const res = await fetch(fallback.url, {
    method: 'POST',
    headers: { 'Content-Type': 'application/json', ...fallback.headers },
```

```
    body: JSON.stringify({ model: fallback.model, messages })
  });
  if (!res.ok) throw new Error(`Fallback LLM also failed: ${res.status}`);
  const data = await res.json();
  return { result: data, provider: `cloud/${fallback.model}` };
}
```

The `provider` field in the return value gets logged to D1, so you can track how often the fallback is used.

Configure in wrangler.toml `[vars]`:
- `LLM_PROVIDER = "selfhosted"` — primary (tried first)
- `LLM_FALLBACK_PROVIDER = "chatllm"` — cloud fallback (or `"grok"`)

---

5. The Python Sandbox — Self-Hosted on Proxmox

The form filler needs Python with:
- `reportlab` — PDF overlay creation with precise coordinates
- `PyPDF2` — PDF merging
- `Pillow` — Signature image processing (RGBA transparency)
- Hebrew text shaping (RTL)

Cloudflare Workers cannot run Python natively. But you have a home server — so we self-host.

5a. Home VM Python API (recommended) ✅

Run a lightweight Python API on a **Proxmox LXC container or VM** at home.

LXC Container Requirements

PDF filler, dashboard, and cloudflared share a single LXC (LLM stays on your dedicated machine):

Requirement	Single service (PDF only)	All-in-one LXC (recommended)
OS	Debian 12 / Ubuntu 22.04	Ubuntu 24.04
CPU	1 vCPU	2–4 vCPU
RAM	512 MB	2 GB (Node.js + Python)
Disk	2 GB	10 GB (OS + Node + Python + assets)
Network	Cloudflare Tunnel (zero port forwarding)	Same
Python	3.10+	3.12
Node.js	—	20 LTS (for dashboard)

> **LXC creation in Proxmox**: Datacenter → Create CT → Template: Ubuntu 24.04 → CPU: 2, RAM: 2048MB, Disk: 10GB → Network: DHCP or static on your LAN. Enable `nesting=1` in Options (needed for systemd services inside LXC).

Setup Steps

```
# 1. Create LXC container in Proxmox (or use an existing VM)
#    Proxmox UI → Create CT → Ubuntu 24.04, 1GB RAM, 5GB disk

# 2. Inside the container:
apt update && apt install -y python3 python3-pip python3-venv

# 3. Create project directory
mkdir -p /opt/form-claw-pdf
cd /opt/form-claw-pdf
python3 -m venv venv
source venv/bin/activate

# 4. Install dependencies
pip install fastapi uvicorn reportlab PyPDF2 Pillow python-multipart

# 5. Copy assets
mkdir -p assets/signatures assets/fonts
# scp or git clone from zeevrussak/form-claw → services/pdf-filler/
# Copy signature PNGs and font TTFs into assets/

# 6. Create systemd service
cat > /etc/systemd/system/form-claw-pdf.service << 'EOF'
[Unit]
Description=Form Claw PDF Filler API
After=network.target

[Service]
Type=simple
User=root
WorkingDirectory=/opt/form-claw-pdf
ExecStart=/opt/form-claw-pdf/venv/bin/uvicorn app:app --host 0.0.0.0 --port 8787
Restart=always
RestartSec=5
Environment=PDF_SERVICE_TOKEN=<your-secret-token>

[Install]
WantedBy=multi-user.target
EOF

systemctl enable --now form-claw-pdf
```

FastAPI Application Skeleton

```python
# /opt/form-claw-pdf/app.py
from fastapi import FastAPI, Header, HTTPException
from pydantic import BaseModel
import base64, os

app = FastAPI(title="Form Claw PDF Filler")
TOKEN = os.environ.get("PDF_SERVICE_TOKEN", "change-me")

class FillRequest(BaseModel):
    original_pdf_b64: str
    fill_instructions: list[dict]    # [{x, y, text, font, size, color}, ...]
    signature_id: str | None = None  # "zeev" or "keren"
    signer_name: str | None = None
    date_str: str | None = None

class FillResponse(BaseModel):
    filled_pdf_b64: str
    pages_processed: int

@app.post("/fill-pdf", response_model=FillResponse)
async def fill_pdf(req: FillRequest, authorization: str = Header(...)):
    if authorization != f"Bearer {TOKEN}":
        raise HTTPException(401, "Unauthorized")
    # ... ReportLab + PyPDF2 fill logic here (ported from fill_fnx_form.py) ...
    return FillResponse(filled_pdf_b64="...", pages_processed=1)

@app.get("/health")
async def health():
    return {"status": "ok"}
```

Exposing the VM to the Internet

Option 1: Cloudflare Tunnel (recommended — zero port forwarding)

```
# Install cloudflared on the VM
curl -L https://github.com/cloudflare/cloudflared/releases/latest/download/
cloudflared-linux-amd64.deb -o cloudflared.deb
dpkg -i cloudflared.deb

# Login and create tunnel
cloudflared tunnel login
cloudflared tunnel create form-claw-pdf

# Configure
cat > /etc/cloudflared/config.yml << EOF
tunnel: <TUNNEL_ID>
credentials-file: /root/.cloudflared/<TUNNEL_ID>.json
ingress:
  - hostname: pdf.savlil.com
    service: http://localhost:8787
  - service: http_status:404
EOF

# DNS: add CNAME pdf.savlil.com → <TUNNEL_ID>.cfargotunnel.com
cloudflared tunnel route dns form-claw-pdf pdf.savlil.com

# Run as service
cloudflared service install
systemctl enable --now cloudflared
```

Result: `https://pdf.savlil.com/fill-pdf` is publicly reachable, TLS terminated by Cloudflare, no ports exposed.

Option 2: Direct with fixed IP
- Port-forward 443 → VM port 8787 on your router
- Use Caddy or Nginx as reverse proxy with Let's Encrypt TLS
- DNS A record: `pdf.savlil.com → <your-fixed-IP>`
- Less secure (IP exposed) but works without Cloudflare Tunnel

Recommendation: Use Cloudflare Tunnel — it's free, secure, and integrates with your existing Cloudflare setup.

Auto-Update from GitHub

```
# /opt/form-claw-pdf/update.sh
#!/bin/bash
cd /opt/form-claw-pdf
git pull origin main
source venv/bin/activate
pip install -r requirements.txt --quiet
systemctl restart form-claw-pdf
echo "Updated at $(date)"
```

Add a cron job or GitHub Actions webhook to trigger updates:

```
# Check for updates every hour
0 * * * * /opt/form-claw-pdf/update.sh >> /var/log/form-claw-update.log 2>&1
```

Or use a GitHub Actions deploy step that SSHs into the VM (see Section 7).

5b. pdf-lib (JavaScript, runs in Worker) — NOT RECOMMENDED

Use pdf-lib (https://pdf-lib.js.org/) for basic PDF manipulation inside the Worker itself.

Limitations:
- ❌ No Hebrew RTL text shaping
- ❌ No transparent PNG overlay with the same quality as ReportLab
- ❌ No ellipse drawing for OR/slash selection

Verdict: Not suitable for the current form complexity.

5c. Cloudflare Workers + Python (experimental) — NOT READY

Cloudflare Python Workers (beta) don't support ReportLab/PyPDF2. Not viable.

---

6. Storage: Split Strategy

Static assets → Home VM local disk (simplest)

Since the Python API runs on your Proxmox VM, static assets live there directly — no need to fetch from R2 on every request:

Asset	VM Path
Ze'ev's signature	`/opt/form-claw-pdf/assets/signatures/zeev.png`
Keren's signature	`/opt/form-claw-pdf/assets/signatures/keren.png`
Hebrew font	`/opt/form-claw-pdf/assets/fonts/FtPilKahol2.ttf`
English font	`/opt/form-claw-pdf/assets/fonts/Playzone.ttf`
Family data	`/opt/form-claw-pdf/assets/family_data.json`

Output PDFs → Cloudflare R2 (optional backup)

Asset	R2 Key
Filled PDFs	`output/<timestamp>/<filename>_filled.pdf`

R2 is useful if you want the dashboard to serve download links for filled PDFs. Otherwise, filled PDFs are attached directly to the Resend reply email and don't need persistent storage.

R2 Cost: Free tier — 10GB storage, 10M reads/month, 1M writes/month. More than enough.

---

7. GitHub Integration & CI/CD

All worker code will be sourced from the **existing GitHub repo** (`zeevrussak/form-claw`).

Repository Structure (proposed additions)

```
form-claw/
├── workers/
│   ├── email-intake/          # existing Cloudflare Email Worker
│   │   ├── src/index.ts
│   │   ├── wrangler.toml
│   │   └── package.json
│   ├── form-processor/        # NEW: orchestrator worker
│   │   ├── src/
│   │   │   ├── index.ts       # main handler
│   │   │   ├── llm.ts         # LLM API client (ChatLLM or Grok)
│   │   │   ├── security.ts    # port of security_filter.py
│   │   │   ├── pdf-service.ts # calls home VM Python API
│   │   │   └── db.ts          # D1 database client
│   │   ├── wrangler.toml
│   │   └── package.json
│   └── analytics/             # OPTIONAL Phase 6: GraphQL analytics
│       ├── src/index.ts
│       ├── wrangler.toml
│       └── package.json
├── services/
│   └── pdf-filler/            # Python API (runs on home Proxmox VM)
│       ├── app.py
│       ├── requirements.txt
│       └── assets/
│           ├── signatures/
│           ├── fonts/
│           └── family_data.json
├── d1/
│   └── schema.sql             # D1 schema (source of truth)
└── nextjs_space/              # existing dashboard (stays on Abacus)
```

Cloudflare Wrangler Configuration

```
# workers/form-processor/wrangler.toml
name = "form-claw-processor"
main = "src/index.ts"
compatibility_date = "2026-01-01"

[[d1_databases]]
binding = "DB"
database_name = "form-claw"
database_id = "<your-d1-database-id>"

[[r2_buckets]]
binding = "ASSETS"
bucket_name = "form-claw-assets"

[vars]
PDF_SERVICE_URL = "https://pdf.savlil.com"
DASHBOARD_URL = "https://formclaw.savlil.com"
LLM_PROVIDER = "selfhosted"           # or "chatllm" or "grok"
LLM_API_URL = "https://llm.savlil.com/v1/chat/completions"
LLM_MODEL = "google/gemma-4-12b-it"      # or your vLLM model name
LLM_FALLBACK_PROVIDER = "chatllm"     # auto-fallback if self-hosted is down
WHITELISTED_SENDERS = "k6622024@gmail.com,2396119@gmail.com"
```

Cloudflare Secrets (via `wrangler secret put`)

Secret Name	Purpose	Source
`RESEND_API_KEY`	Send reply emails	Resend dashboard
`LLM_API_KEY`	LLM API authentication	See Option A or B below
`PDF_SERVICE_TOKEN`	Auth for home VM Python API	Self-generated (e.g., `openssl rand -hex 32`)
`HEARTBEAT_TOKEN`	Dashboard health reporting	Existing value

Cloudflare Variables (non-secret, in wrangler.toml `[vars]`)

Variable	Value
`PDF_SERVICE_URL`	`https://pdf.savlil.com`
`DASHBOARD_URL`	`https://formclaw.savlil.com`
`LLM_PROVIDER`	`selfhosted`, `chatllm`, or `grok`
`LLM_API_URL`	`https://llm.savlil.com/v1/chat/completions` (for self-hosted)
`LLM_MODEL`	`gemma4:12b` (or `qwen2.5-vl:7b`, etc.)
`LLM_FALLBACK_PROVIDER`	`chatllm` (used if primary is down)
`WHITELISTED_SENDERS`	`k6622024@gmail.com,2396119@gmail.com,...`

CI/CD via GitHub Actions

```yaml
# .github/workflows/deploy-workers.yml
name: Deploy Cloudflare Workers
on:
  push:
    branches: [main]
    paths: ['workers/**']
jobs:
  deploy:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: cloudflare/wrangler-action@v3
        with:
          apiToken: ${{ secrets.CLOUDFLARE_API_TOKEN }}
          workingDirectory: workers/form-processor

# .github/workflows/deploy-pdf-service.yml
name: Deploy PDF Service to Home VM
on:
  push:
    branches: [main]
    paths: ['services/pdf-filler/**']
jobs:
  deploy:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - name: Deploy via SSH
        uses: appleboy/ssh-action@v1
        with:
          host: pdf.savlil.com  # or your fixed IP
          username: root
          key: ${{ secrets.VM_SSH_PRIVATE_KEY }}
          script: |
            cd /opt/form-claw-pdf
            git pull origin main
            source venv/bin/activate
            pip install -r requirements.txt --quiet
            systemctl restart form-claw-pdf
            echo "Deployed at $(date)"
```

GitHub Secrets Required

Secret	Purpose
`CLOUDFLARE_API_TOKEN`	Deploy Workers via Wrangler
`VM_SSH_PRIVATE_KEY`	SSH deploy to Proxmox VM

---

8. Cost Estimate — 2 forms/day

Per-form processing breakdown

Each form fill involves approximately:
- **1 LLM vision call** (analyze 1-2 PDF pages as images): ~2,000 input tokens + ~1,500 output tokens per page
- **1 LLM text call** (generate fill instructions): ~3,000 input tokens + ~2,000 output tokens
- **1 Python API call** (fill PDF): CPU time ~3 seconds
- **1 Resend email** (reply with PDF)
- **1 DB write** (log entry)

Estimated tokens per form: ~5,000 input + ~3,500 output (single page form)

Monthly cost at 2 forms/day (60 forms/month)

Component	Option A: ChatLLM	Option B: Grok (xAI)	Option C: Self-Hosted LLM
LLM API	Included in Abacus sub	~$2.70/mo	$0 (your hardware)
Cloudflare Workers	Free tier	Free tier	Free tier
Cloudflare D1	Free tier (5GB)	Free tier	Free tier
Cloudflare R2	Free tier (10GB)	Free tier	Free tier
Cloudflare Email Routing	Free	Free	Free
Resend API	Free tier	Free tier	Free tier
Home VM (PDF + Dashboard)	$0 (your HW)	$0 (your HW)	$0 (your HW)
Home VM (Ollama + GPU)	—	—	~$3-8/mo electricity
GitHub	Free	Free	Free
Total monthly	**~$0/mo**	**~$3/mo**	**~$3-8/mo** (electricity only)
Total one-time	$0	$0	$0–$400 (GPU if buying new)

Token math for Grok

```
Per form:
  Vision: 2,000 input × $2/1M = $0.004  +  1,500 output × $10/1M = $0.015
  Text:   3,000 input × $2/1M = $0.006  +  2,000 output × $10/1M = $0.020
  Total per form: ~$0.045

Monthly (60 forms): 60 × $0.045 = $2.70
```

Note: Multi-page forms or complex forms requiring multiple LLM rounds could double costs. Estimate assumes typical 1-page Israeli government/HMO forms.

---

9. Implementation Phases

Phase 1: Foundation (1-2 days)

- [ ] Create Cloudflare D1 database with schema (see Section 3.3)
- [ ] Create R2 bucket (for optional output PDF backup)
- [ ] Set up GitHub repo structure (`workers/` , `services/`)
- [ ] Set up GitHub Actions for Cloudflare + VM deployment

Phase 2: Home VM Setup (1-2 days)

- [ ] Provision LXC container on Proxmox (Ubuntu 24.04, 1GB RAM, 5GB disk)
- [ ] Install Python 3.12, create venv, install deps (FastAPI, ReportLab, PyPDF2, Pillow)
- [ ] Port `fill_fnx_form.py` + skill logic into stateless FastAPI `/fill-pdf` endpoint
- [ ] Copy assets (signatures, fonts, family_data.json) into `/opt/form-claw-pdf/assets/`
- [ ] Set up Cloudflare Tunnel (`pdf.savlil.com` → VM:8787)
- [ ] Set up systemd service for auto-start
- [ ] Test with known form + fill instructions via curl

Phase 3: Processor Worker (2-3 days)

- [ ] Write `form-claw-processor` Worker:
- Webhook handler
- Security filter (port `security_filter.py` to TypeScript)
- LLM client (with provider switch: ChatLLM / Grok)
- PDF service client (calls `https://pdf.savlil.com/fill-pdf`)
- Resend client (port `send_resend_reply.py` logic)
- D1 logging (`env.DB.prepare()` statements)
- Heartbeat reporting
- [ ] Deploy and wire up email intake Worker

Phase 4: Dashboard Self-Hosting (1-2 days)

- [ ] Install Node.js 20 on the VM (same LXC or sibling)
- [ ] Replace Prisma with D1 HTTP client in all API routes (~10 files)
- [ ] Set up NextAuth with local SQLite or Cloudflare Access
- [ ] Build and deploy via systemd service
- [ ] Add `formclaw.savlil.com` ingress to Cloudflare Tunnel

- [ ] Update health check endpoint for new architecture
- [ ] Test all dashboard pages with D1 data

Phase 5: Cutover & Monitoring (1 day)

- [ ] Update email intake Worker's `WEBHOOK_URL` to processor Worker
- [ ] Run end-to-end test (email → fill → reply)
- [ ] Monitor first 24h of production
- [ ] Disable old Abacus daemon

Phase 6 (optional): Self-Hosted LLM

- [ ] Install Ollama on the VM (or a dedicated GPU-passthrough VM)
- [ ] Pull Gemma 4 12B (or Qwen2.5-VL-7B for better Hebrew)
- [ ] Set up GPU passthrough if using dedicated GPU
- [ ] Add `llm.savlil.com` ingress to Cloudflare Tunnel (or route internally)
- [ ] Test with 10+ real Hebrew forms, compare quality vs ChatLLM
- [ ] Switch `LLM_PROVIDER` to `selfhosted` in wrangler.toml if quality is acceptable
- [ ] Configure fallback: if self-hosted is down, retry with ChatLLM

Total estimated effort: 7-11 days (8-12 with Phase 6)

---

10. Risks & Mitigations

Risk	Impact	Mitigation
Worker 30s CPU limit exceeded on complex forms	Processing fails	Use Cloudflare Workers Unbound (no CPU limit, $0.02/M requests) or offload heavy work to Python service
Hebrew RTL quality degrades with Grok vs GPT-4o	Incorrect field fills	Test both providers with 5+ real forms before switching; keep ChatLLM as fallback
Home VM downtime (power outage, Proxmox reboot)	Processing queued/fails	UPS for Proxmox host; Worker retries with exponential backoff; email stays in Cloudflare queue
Home internet outage	VM unreachable	Cloudflare Tunnel auto-reconnects; forms queue in Worker until VM is back; set 3-retry with 30s delay
Self-hosted LLM Hebrew quality	Incorrect field mapping	Test 10+ real forms; keep ChatLLM as `LLM_FALLBACK_PROVIDER`; hybrid mode possible (self-hosted for English, ChatLLM for Hebrew)
GPU memory exhaustion	Ollama OOM	Use quantized model (Q4); limit concurrent requests to 1; set `OLLAMA_MAX_LOADED_MODELS=1`
PDF page-to-image conversion needed for vision	Workers can’t render PDFs	Do PDF→PNG conversion in the home VM Python service; return images to Worker for LLM call
D1 schema drift from Prisma schema	Dashboard data mismatch	Maintain single source-of-truth SQL migration files; apply to both D1 and Prisma schema in same PR

11. Decision Matrix

Question	Recommended Choice
LLM Provider	**Self-hosted Gemma 4 12B** (free, private). Fall back to **ChatLLM** or **Grok** if quality issues.
PDF Processing	**Self-hosted Python API on Proxmox VM** via Cloudflare Tunnel
Database	**Cloudflare D1** (SQLite, free tier, native to Workers)
Storage (assets)	**Local disk on home VM** (signatures, fonts, model weights)
Storage (output)	**Cloudflare R2** (optional backup for filled PDFs)
Dashboard	**Self-hosted on Proxmox VM** via Cloudflare Tunnel
CI/CD	**GitHub Actions → Wrangler** (Workers) + **SSH deploy** (VM)
Analytics API	**Cloudflare D1 + GraphQL** (see Section 14)

---

12. Analytics via GraphQL (optional enhancement)

The current dashboard reads stats via REST endpoints (`/api/stats` , `/api/stats/range`). With D1 as the data source, you could expose a **GraphQL API** on a Cloudflare Worker for analytics — this is a good fit because:

Why GraphQL works well here

- **Flexible queries**: The dashboard already supports date ranges, sender filters, target-person filters, and status filters. GraphQL lets the frontend request exactly the aggregations it needs in a single request instead of multiple REST calls.
- **D1 is SQLite**: GraphQL resolvers map cleanly to SQL queries. No ORM needed — just `env.DB.prepare()` with parameterized queries.
- **Edge performance**: A Worker + D1 GraphQL endpoint runs at the edge with ~0ms DB latency.
- **Schema-first**: GraphQL's typed schema documents the analytics API automatically.

Example GraphQL Schema

```
type Query {
  stats(startDate: String, endDate: String): StatsOverview!
  dailyStats(startDate: String!, endDate: String!): [DailyStat!]!
  logs(page: Int, limit: Int, status: String, sender: String, search: String):
LogPage!
  senderBreakdown(startDate: String, endDate: String): [SenderStat!]!
  targetBreakdown(startDate: String, endDate: String): [TargetStat!]!
}

type StatsOverview {
  totalForms: Int!
  successCount: Int!
  failureCount: Int!
  successRate: Float!
  avgProcessingTime: Float
  todayCount: Int!
}

type DailyStat {
  date: String!
  total: Int!
  success: Int!
  failure: Int!
}

type LogPage {
  logs: [FormLog!]!
  total: Int!
  page: Int!
  totalPages: Int!
}

type FormLog {
  id: String!
  senderEmail: String
  subject: String
  receivedAt: String!
  processingStatus: String!
  processingTime: Float
  targetPerson: String
  errorType: String
  errorMessage: String
}

type SenderStat { sender: String!, count: Int! }
type TargetStat { target: String!, count: Int! }
```

Implementation

Use `graphql-yoga` (https://the-guild.dev/graphql/yoga-server) — it runs natively in Cloudflare Workers:

```
// workers/analytics/src/index.ts
import { createYoga, createSchema } from 'graphql-yoga';

export default {
  fetch(request: Request, env: Env) {
    const yoga = createYoga({
      schema: createSchema({
        typeDefs,
        resolvers: {
          Query: {
            stats: async (_, { startDate, endDate }) => {
              const result = await env.DB.prepare(
                `SELECT COUNT(*) as total,
                  SUM(CASE WHEN processing_status='success' THEN 1 ELSE 0 END) as success,
                  SUM(CASE WHEN processing_status='failed' THEN 1 ELSE 0 END) as failure,
                  AVG(processing_time_seconds) as avg_time
                FROM form_processing_log
                WHERE received_at BETWEEN ?1 AND ?2`
              ).bind(startDate || '2000-01-01', endDate || '2099-12-31').first();
              return { ...result, successRate: result.total ? result.success / result.total : 0 };
            },
            // ... other resolvers
          }
        }
      })
    });
    return yoga.fetch(request, env);
  }
};
```

When to add GraphQL

Not in Phase 1. Get the core form-filling pipeline working first. GraphQL is a Phase 6 enhancement — add it once:

1. D1 is populated with real data
2. The dashboard needs more flexible analytics (or you want a public/mobile analytics view)
3. You want to query analytics from multiple clients (dashboard, mobile, CLI)

For Phase 1-5, the existing REST endpoints on the Abacus dashboard are sufficient.

---

13. What Stays the Same

- ✅ Email intake Cloudflare Worker (already deployed)
- ✅ Resend for outbound email (already configured)
- ✅ GitHub repository as source of truth
- ✅ Sender whitelist logic
- ✅ Security filter logic (ported to TypeScript)
- ✅ Family data structure (family_data.json)
- ✅ Signature assets (PNGs with transparency)
- ✅ Form-filling skill logic (coordinate placement, ellipses, split-digit fields)

14. What Changes

- 🔄 Form processor: Abacus AI daemon → Cloudflare Worker + home VM Python API
- 🔄 Database: Abacus-hosted PostgreSQL → Cloudflare D1 (SQLite)
- 🔄 Asset storage: Abacus VM filesystem → Home VM local disk
- 🔄 LLM calls: Abacus built-in agent → Self-hosted Gemma 4 / Ollama (or ChatLLM / Grok as fall-back)
- 🔄 Code execution: Abacus agent Python sandbox → Self-hosted FastAPI on Proxmox
- 🔄 Dashboard: Abacus-hosted → Self-hosted on Proxmox VM via Cloudflare Tunnel
- 🔄 Analytics (future): REST → GraphQL on Cloudflare Worker